

QEMU MicroVM Mode + GUI - Complete Setup Guide

Goal: Run Chromium browser in QEMU microVM with native GUI display

Requirements:

- Ubuntu 22.04 LTS or later
 - KVM support (Intel VT-x or AMD-V)
 - 8GB RAM minimum
 - QEMU 6.2+ (we'll install latest)
-

Part 1: System Preparation

Step 1.1: Verify KVM Support

```
bash

# Check virtualization
egrep -c '(vmx|svm)' /proc/cpuinfo
# Should return > 0

# Check KVM device
ls -l /dev/kvm

# Load KVM modules
sudo modprobe kvm
sudo modprobe kvm_intel # OR: sudo modprobe kvm_amd

# Add user to kvm group
sudo usermod -aG kvm $USER
newgrp kvm
```

Step 1.2: Install QEMU (Latest Version)

```
bash
```

```
# Update system
```

```
sudo apt update && sudo apt upgrade -y
```

```
# Install QEMU and dependencies
```

```
sudo apt install -y \  
    qemu-system-x86 \  
    qemu-utils \  
    qemu-kvm \  
    libvirt-daemon-system \  
    libvirt-clients \  
    bridge-utils \  
    virt-manager \  
    ovmf \  
    cloud-image-utils \  
    genisoimage
```

```
# Verify QEMU version
```

```
qemu-system-x86_64 --version
```

```
# Should show: QEMU emulator version 6.2.0 or higher
```

```
# If version is too old, build from source:
```

```
# (Optional - skip if version is 6.2+)
```

Step 1.3: Build Latest QEMU (Optional)

```
bash
```

```
# Only if your QEMU version is < 6.2
```

```
cd ~
```

```
mkdir qemu-build && cd qemu-build
```

```
# Install build dependencies
```

```
sudo apt install -y \  
    build-essential \  
    ninja-build \  
    libglib2.0-dev \  
    libpixman-1-dev \  
    libsdl2-dev \  
    libgtk-3-dev \  
    libspice-server-dev \  
    libspice-protocol-dev \  
    libusb-1.0-0-dev
```

```
# Download QEMU source
```

```
wget https://download.qemu.org/qemu-8.2.0.tar.xz
```

```
tar xf qemu-8.2.0.tar.xz
```

```
cd qemu-8.2.0
```

```
# Configure and build
```

```
./configure --enable-kvm --enable-sdl --enable-gtk --enable-spice
```

```
make -j$(nproc)
```

```
sudo make install
```

```
# Verify
```

```
qemu-system-x86_64 --version
```

Part 2: Create Guest OS Image

Step 2.1: Download Ubuntu Cloud Image

```
bash
```

```
# Create working directory
```

```
mkdir -p ~/qemu-microvm-setup
```

```
cd ~/qemu-microvm-setup
```

```
# Download Ubuntu 22.04 cloud image
```

```
wget https://cloud-images.ubuntu.com/jammy/current/jammy-server-cloudimg-amd64.img
```

```
# Create working copy
```

```
cp jammy-server-cloudimg-amd64.img ubuntu-microvm.img
```

```
# Resize image
```

```
qemu-img resize ubuntu-microvm.img +15G
```

```
echo "✅ Base image created: ubuntu-microvm.img"
```

Step 2.2: Create Cloud-Init Configuration

bash

cd ~/qemu-microvm-setup

Create user-data for cloud-init

cat > user-data << 'EOF'

#cloud-config

hostname: qemu-browser

users:

- name: browser
- sudo: ALL=(ALL) NOPASSWD:ALL
- groups: users, admin
- home: /home/browser
- shell: /bin/bash
- lock_passwd: false
- # Password: browser123
- passwd: \$6\$rounds=4096\$saltsalt\$L7gFZEqSLEq9mNwsKNtA7.F3dXQdpI4jHKEFLn3M7VTdKR8G

packages:

- ubuntu-desktop-minimal
- chromium-browser
- firefox
- xserver-xorg
- xinit
- lightdm
- openssh-server
- net-tools
- inotify-tools
- qemu-guest-agent

runcmd:

- systemctl enable lightdm
- systemctl set-default graphical.target
- systemctl enable qemu-guest-agent
- systemctl start qemu-guest-agent

chpasswd:

list: |

- root:qemu123
- browser:browser123

expire: False

growpart:

mode: auto

devices: ['/']

EOF

```
# Create meta-data
cat > meta-data << 'EOF'
instance-id: qemu-microvm-001
local-hostname: qemu-browser
EOF

# Generate cloud-init ISO
cloud-localds cloud-init.img user-data meta-data

echo "✅ Cloud-init ISO created"
```

Part 3: Set Up Networking

Step 3.1: Create Network Bridge (Method 1)

```
bash

cat > ~/qemu-microvm-setup/setup-bridge.sh << 'EOF'
#!/bin/bash

# Create bridge
sudo ip link add br0 type bridge
sudo ip link set br0 up
sudo ip addr add 192.168.122.1/24 dev br0

# Enable IP forwarding
sudo sysctl -w net.ipv4.ip_forward=1

# Get main interface
MAIN_IF=$(ip route | grep default | awk '{print $5}')

# NAT setup
sudo iptables -t nat -A POSTROUTING -o $MAIN_IF -j MASQUERADE
sudo iptables -A FORWARD -m conntrack --ctstate RELATED,ESTABLISHED -j ACCEPT
sudo iptables -A FORWARD -i br0 -o $MAIN_IF -j ACCEPT

echo "✅ Bridge network configured: br0 at 192.168.122.1"
EOF

chmod +x setup-bridge.sh
```

Step 3.2: Create TAP Device (Method 2 - Simpler)

bash

```
cat > ~/qemu-microvm-setup/setup-network.sh << 'EOF'
#!/bin/bash

# Create TAP device
sudo ip tuntap add tap-qemu mode tap user $(whoami)
sudo ip link set tap-qemu up
sudo ip addr add 192.168.100.1/24 dev tap-qemu

# Enable forwarding
sudo sysctl -w net.ipv4.ip_forward=1

# NAT
MAIN_IF=$(ip route | grep default | awk '{print $5}')
sudo iptables -t nat -A POSTROUTING -o $MAIN_IF -j MASQUERADE
sudo iptables -A FORWARD -i tap-qemu -j ACCEPT
sudo iptables -A FORWARD -o tap-qemu -j ACCEPT

echo "✅ TAP network ready: tap-qemu at 192.168.100.1"
EOF

chmod +x setup-network.sh

# Run it
./setup-network.sh
```

Part 4: QEMU MicroVM Configuration

Step 4.1: Create MicroVM Launch Script (GTK Display)

bash

```
cat > ~/qemu-microvm-setup/launch-microvm-gtk.sh << 'EOF'
#!/bin/bash

echo "🚀 Starting QEMU MicroVM with GTK display..."

# Setup network
./setup-network.sh

# Start QEMU in microVM mode
qemu-system-x86_64 \
    -M microvm,x-option-roms=off,pit=off,pic=off,rtc=on \
    -enable-kvm \
    -cpu host \
    -smp 2 \
    -m 2G \
    -drive file=ubuntu-microvm.img,if=none,id=root,format=qcow2 \
    -device virtio-blk-device,drive=root \
    -drive file=cloud-init.img,if=none,id=cidata,format=raw,readonly=on \
    -device virtio-blk-device,drive=cidata \
    -netdev tap,id=net0,ifname=tap-qemu,script=no,downscript=no \
    -device virtio-net-device,netdev=net0 \
    -device virtio-gpu-pci \
    -display gtk,gl=on \
    -serial stdio \
    -nodefaults \
    -no-user-config \
    -nographic=false

echo "✅ QEMU MicroVM stopped"
EOF

chmod +x launch-microvm-gtk.sh
```

Step 4.2: Create MicroVM Launch Script (SDL Display)

bash

```
cat > ~/qemu-microvm-setup/launch-microvm-sdl.sh << 'EOF'
#!/bin/bash

echo "🚀 Starting QEMU MicroVM with SDL display..."

./setup-network.sh

qemu-system-x86_64 \
    -M microvm,x-option-roms=off,pit=off,pic=off,rtc=on \
    -enable-kvm \
    -cpu host \
    -smp 2 \
    -m 2G \
    -drive file=ubuntu-microvm.img,if=none,id=root,format=qcow2 \
    -device virtio-blk-device,drive=root \
    -drive file=cloud-init.img,if=none,id=cidata,format=raw,readonly=on \
    -device virtio-blk-device,drive=cidata \
    -netdev tap,id=net0,ifname=tap-qemu,script=no,downscript=no \
    -device virtio-net-device,netdev=net0 \
    -device virtio-gpu-pci \
    -display sdl,gl=on \
    -vga virtio

echo "✅ VM stopped"
EOF

chmod +x launch-microvm-sdl.sh
```

Step 4.3: Create Standard Launch (VNC - No GPU)

bash

```
cat > ~/qemu-microvm-setup/launch-microvm-vnc.sh << 'EOF'
#!/bin/bash

echo "🚀 Starting QEMU MicroVM with VNC..."

./setup-network.sh

qemu-system-x86_64 \
    -M microvm,x-option-roms=off,pit=off,pic=off,rtc=on \
    -enable-kvm \
    -cpu host \
    -smp 2 \
    -m 2G \
    -drive file=ubuntu-microvm.img,if=none,id=root,format=qcow2 \
    -device virtio-blk-device,drive=root \
    -drive file=cloud-init.img,if=none,id=cidata,format=raw,readonly=on \
    -device virtio-blk-device,drive=cidata \
    -netdev tap,id=net0,ifname=tap-qemu,script=no,downscript=no \
    -device virtio-net-device,netdev=net0 \
    -vnc :0 \
    -vga std &

QEMU_PID=$!

echo "⌚ Waiting for VM boot..."
sleep 10

echo "🌐 Connecting via VNC..."
vncviewer localhost:5900 &

wait $QEMU_PID
EOF

chmod +x launch-microvm-vnc.sh
```

Step 4.4: Create SPICE Launch Script

bash

```
cat > ~/qemu-microvm-setup/launch-microvm-spice.sh << 'EOF'
#!/bin/bash

echo "🚀 Starting QEMU MicroVM with SPICE..."

./setup-network.sh

qemu-system-x86_64 \
    -M microvm,x-option-roms=off,pit=off,pic=off,rtc=on \
    -enable-kvm \
    -cpu host \
    -smp 2 \
    -m 2G \
    -drive file=ubuntu-microvm.img,if=none,id=root,format=qcow2 \
    -device virtio-blk-device,drive=root \
    -drive file=cloud-init.img,if=none,id=cidata,format=raw,readonly=on \
    -device virtio-blk-device,drive=cidata \
    -netdev tap,id=net0,ifname=tap-qemu,script=no,downscript=no \
    -device virtio-net-device,netdev=net0 \
    -device virtio-vga \
    -spice port=5900,disable-ticketing=on \
    -device virtio-serial \
    -chardev spicevmc,id=vdagent,debug=0,name=vdagent \
    -device virtserialport,chardev=vdagent,name=com.redhat.spice.0 &

QEMU_PID=$!

echo "⌚ VM starting..."
sleep 10

echo "🌐 Opening SPICE client..."
remote-viewer spice://localhost:5900 &

wait $QEMU_PID
EOF

chmod +x launch-microvm-spice.sh
```

Part 5: First Boot and Configuration

Step 5.1: Initial Boot

```
bash

cd ~/qemu-microvm-setup

# First boot with GTK (easiest to see console)
./launch-microvm-gtk.sh

# Wait for cloud-init to complete (2-3 minutes)
# You should see desktop login screen

# Login: browser / browser123
```

Step 5.2: Configure Inside VM (If Needed)

```
bash

# If desktop doesn't auto-start, SSH in:
ssh browser@192.168.100.2
# Password: browser123

# Install additional packages
sudo apt update
sudo apt install -y \
    chromium-browser \
    firefox \
    vlc \
    gedit

# Configure auto-login (optional)
sudo mkdir -p /etc/lightdm/lightdm.conf.d/
sudo tee /etc/lightdm/lightdm.conf.d/50-autologin.conf << EOF
[Seat:*]
autologin-user=browser
autologin-user-timeout=0
EOF

# Reboot to apply
sudo reboot
```

Part 6: File Monitoring

Step 6.1: Host-Side Monitor

bash

```
cat > ~/qemu-microvm-setup/file-monitor.py << 'EOF'
#!/usr/bin/env python3
"""
Monitor files in QEMU microVM
"""

import subprocess
import logging
import time

logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(message)s'
)

def monitor_vm():
    vm_ip = "192.168.100.2"
    watch_path = "/home/browser/Downloads"

    logging.info(f"👁 Monitoring {vm_ip}:{watch_path}")

    # Monitor via SSH
    cmd = f"""
        ssh -o StrictHostKeyChecking=no browser@{vm_ip} \
        'inotifywait -m {watch_path} -e create -e modify -e delete'
    """

    proc = subprocess.Popen(
        cmd,
        shell=True,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )

    try:
        for line in proc.stdout:
            if line.strip():
                logging.warning(f"🚨 {line.strip()}")
    except KeyboardInterrupt:
        logging.info("Stopped")
        proc.kill()

if __name__ == "__main__":
    monitor_vm()
EOF
```

Part 7: Desktop Integration

Step 7.1: Create Desktop Launchers

```
bash

# GTK version
cat > ~/.local/share/applications/qemu-microvm-gtk.desktop << EOF
[Desktop Entry]
Version=1.0
Type=Application
Name=Isolated Browser (QEMU MicroVM GTK)
Comment=Launch browser in QEMU microVM with GTK display
Exec=$HOME/qemu-microvm-setup/launch-microvm-gtk.sh
Icon=computer
Terminal=true
Categories=System;Emulator;
EOF

# SPICE version
cat > ~/.local/share/applications/qemu-microvm-spice.desktop << EOF
[Desktop Entry]
Version=1.0
Type=Application
Name=Isolated Browser (QEMU SPICE)
Comment=Launch browser with SPICE display
Exec=$HOME/qemu-microvm-setup/launch-microvm-spice.sh
Icon=computer
Terminal=true
Categories=System;Emulator;
EOF

update-desktop-database ~/.local/share/applications/
```

Part 8: Performance Benchmarking

Step 8.1: Boot Time Test

bash

```
cat > ~/qemu-microvm-setup/benchmark.sh << 'EOF'
#!/bin/bash

echo "🕒 Benchmarking QEMU MicroVM boot time..."

./setup-network.sh

START=$(date +%s%3N)

qemu-system-x86_64 \
  -M microvm \
  -enable-kvm \
  -cpu host \
  -smp 2 \
  -m 2G \
  -drive file=ubuntu-microvm.img,if=none,id=root \
  -device virtio-blk-device,drive=root \
  -netdev tap,id=net0,ifname=tap-qemu,script=no \
  -device virtio-net-device,netdev=net0 \
  -nographic &

QEMU_PID=$!

# Wait for network
while ! ping -c 1 192.168.100.2 &>/dev/null; do
  sleep 0.1
done

END=$(date +%s%3N)
BOOT_TIME=$((END - START))

echo "✅ QEMU MicroVM boot: ${BOOT_TIME}ms"

kill $QEMU_PID
EOF

chmod +x benchmark.sh
```

Part 9: Advanced Features

Step 9.1: Snapshot Support

```
bash
```

```
# Create snapshot
```

```
qemu-img snapshot -c clean-state ubuntu-microvm.img
```

```
# List snapshots
```

```
qemu-img snapshot -l ubuntu-microvm.img
```

```
# Restore snapshot
```

```
qemu-img snapshot -a clean-state ubuntu-microvm.img
```

```
# Delete snapshot
```

```
qemu-img snapshot -d clean-state ubuntu-microvm.img
```

Step 9.2: Monitor Interface

```
bash
```

```
# Launch with QEMU monitor
```

```
qemu-system-x86_64 \  
  -M microvm \  
  -enable-kvm \  
  -cpu host \  
  -smp 2 \  
  -m 2G \  
  -drive file=ubuntu-microvm.img,if=none,id=root \  
  -device virtio-blk-device,drive=root \  
  -monitor stdio
```

```
# In monitor, you can:
```

```
# info status - VM status
```

```
# info network - Network info
```

```
# stop - Pause VM
```

```
# cont - Resume VM
```

```
# quit - Exit
```

Configuration Files Summary


```
qemu-microvm-setup/  
├─ ubuntu-microvm.img          # VM disk  
├─ cloud-init.img              # Cloud-init config  
├─ user-data                    # Cloud-init user data  
├─ meta-data                   # Cloud-init metadata  
├─ setup-network.sh            # Network setup  
├─ launch-microvm-gtk.sh       # GTK launcher  
├─ launch-microvm-sdl.sh       # SDL launcher  
├─ launch-microvm-vnc.sh       # VNC launcher  
├─ launch-microvm-spice.sh     # SPICE launcher  
├─ file-monitor.py             # File monitor  
└─ benchmark.sh                # Performance test
```

Quick Start

```
bash  
  
# 1. Setup network (first time)  
cd ~/qemu-microvm-setup  
./setup-network.sh  
  
# 2. Launch with GTK (best option)  
./launch-microvm-gtk.sh  
  
# 3. Or launch with SPICE (better performance)  
./launch-microvm-spice.sh  
  
# 4. Monitor files  
python3 file-monitor.py  
  
# 5. Benchmark  
./benchmark.sh
```

Expected Performance

- **Boot Time:** 400-600ms (microVM mode)
 - **Full System Ready:** ~3-5 seconds
 - **Display:** Native (GTK/SDL) - no VNC lag
 - **Memory:** ~2GB RAM usage
-

Advantages & Disadvantages

✅ Advantages:

- **BEST GUI support** (GTK, SDL, SPICE native)
- GPU acceleration works
- Very mature and stable
- Excellent documentation
- Own kernel (hardware isolation)
- Snapshot/restore support

❌ Disadvantages:

- Slower boot than Firecracker (400ms vs 125ms)
 - Larger code base (bigger attack surface)
 - More resource overhead
-

Troubleshooting

No display:

```
bash

# Check if virtio-gpu is supported
lsmmod | grep virtio

# Try different display backends
-display gtk
-display sdl
-display vnc=:0
```

Network not working:

```
bash

# Verify TAP device
ip addr show tap-qemu

# Check VM can reach gateway
ssh browser@192.168.100.2
ping 192.168.100.1
```

This completes QEMU MicroVM setup! 🎉